Loop

▸ Syntax

⟨Programm⟩ ::= ε | ⟨Befehl⟩ "↲" ⟨Programm⟩
⟨Befehl⟩ ::= "++" ⟨Variable⟩
| "−−" ⟨Variable⟩
| "*loop* " ⟨Variable⟩ "↲" ⟨Programm⟩ "*end*"
| "*stop*"

⟨Variable⟩ : String aus Zahlen und Buchstaben
aber nicht "*loop*", "*end*" oder "*stop*"

↲ : Zeilenwechsel

▸ Semantik

- Die Befehle eines Programms werden nacheinander abgearbeitet.
- Ein Programm verwendet eine beliebige Anzahl von Variablen, die beliebige Zahlen $\in \mathbb{N}_0$ speichern können.
- Am Anfang seien alle Variablen = 0.

"Registermaschine"
"Random Access Machine"

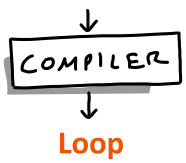
++x	Variable x wird um 1 erhöht
−−x	Wenn x = 0 ist, geschieht nichts. Ansonsten wird x um 1 erniedrigt.
loop x ⟨Programm⟩ *end*	Führt wiederholt ⟨Programm⟩ aus, und zwar genau so oft, wie der Wert in x am Anfang war.
stop	Stoppt das Programm

▸ Beispiel:

Das folgende Programm verdoppelt den Inhalt der Variablen x:

```
loop x
   ++x
end
```

extended Loop



Loop

▸ Variablen löschen

$x \leftarrow 0$

```
loop x
    --x
end
```

▸ Variable um Konstante $c \in \mathbb{N}$ erhöhen:

$x \leftarrow x + c$

```
++x
 ⋮   } c mal
++x
```

▸ Variable auf Konstante $c \in \mathbb{N}$ setzen:

$x \leftarrow c$

```
x ← 0
x ← x + c
```

▸ Variable um Wert einer Variablen erhöhen:

$x \leftarrow x + y$
$x \leftarrow y + x$

```
loop y
    ++x
end
```

▸ Variable in eine Variable kopieren:

$x \leftarrow y$

```
x ← 0
x ← x + y
```

▸ Summe aus Variable und Konstante in zweite Variable schreiben:

$x \leftarrow y + c$
$x \leftarrow c + y$

```
x ← y
x ← x + c
```

▸ Summe zweier Variablen in dritte Variable schreiben:

$x \leftarrow y + z$

```
x ← y
x ← x + z
```

▸ Subtraktion:

$x \leftarrow x - c$
$x \leftarrow x - y$
$x \leftarrow y - x$
$x \leftarrow y - c$
$x \leftarrow y - z$

▸ Multiplikation:

$x \leftarrow y \cdot z$

```
x ← 0
loop z
    x ← x + y
end
```

▸ Potenzieren:

x ← y ^ z

```
x ← 1
loop z
   x ← x · y
end
```

▸ Bedingte Anweisung

Wert = 0 ⇔ *false*
Wert > 0 ⇔ *true*

```
if x
   ⟨P1⟩
else
   ⟨P2⟩
end
```

```
a ← 1
loop x
   −−a
end
b ← 1
loop a
   −−b
   ⟨P2⟩
end
loop b
   ⟨P1⟩
end
```

a und b kommen nicht in ⟨P1⟩ und ⟨P2⟩ vor.

```
if x
   ⟨P1⟩
end
```

```
if x
   ⟨P1⟩
else
end
```

▸ Wahrheitswerte

x ← *true*

```
x ← 1
```

x ← *false*

```
x ← 0
```

▸ logische Operationen

x ← *not* y

```
if y
   x ← false
else
   x ← true
end
```

x ← y *and* z

```
x ← false
if y
   if z
      x ← true
   end
end
```

x ← y *or* z

```
x ← y + z
```

▸ Vergleich

x ← y ≠ z

```
a ← y − z
b ← z − y
x ← a or b
```

x ← y = z

```
x ← y ≠ z
x ← not x
```

▸ Ganzzahldivision

x ← y / z

```
x ← 0
a ← y
++a
loop y
   a ← a − z
   if a
      ++x
   end
end
```

▸ Modulo = Rest einer Ganzzahl-Division

x ← y % z

```
x ← y / z
x ← x · z
x ← y − x
```

▸ Kodierung von Strings als Zahl

Achtung: "0" ≠ "000"

Alphabet $\Sigma = \{c_1, c_2, \ldots, c_m\}$
Kodiere jedes c_j durch die Zahl j:
$code(c_j) = j$ für alle $j \in \{1, 2, \ldots, m\}$

Sei $a_0a_1\ldots a_n \in \Sigma^*$, d.h. $a_i \in \Sigma$ für $i \in \{0, 1, \ldots, n\}$

$$code(a_0a_1\ldots a_n) = \sum_{i=0}^{n} code(a_i) \cdot m^i$$

$code(\varepsilon) = 0$

Beispiel:
$\Sigma = \{A, B, C\}$, $m = 3$
code(A) = 1
code(B) = 2
code(C) = 3

$$code(BCA) = \underbrace{code(B)}_{2} + \underbrace{code(C)}_{3} \cdot 3 + \underbrace{code(A)}_{1} \cdot 3^2 = 20$$

String s	code(s)
ε	0
A	1
B	2
C	3
AA	4
BA	5
CA	6
AB	7
BB	8
CB	9
AC	10
BC	11
CC	12
AAA	13
BAA	14
CAA	15
ABA	16
BBA	17
CBA	18
ACA	19
BCA	20
⋮	⋮

$$\text{code}(a_0a_1\ldots a_n) = \sum_{i=0}^{n} \text{code}(a_i) \cdot m^i$$

$$\text{code(BCA)} = \underbrace{\text{code(B)}}_{2} + \underbrace{\text{code(C)}}_{3} \cdot 3 + \underbrace{\text{code(A)}}_{1} \cdot 3^2 = 20$$

▸ Erstes Zeichen im String

$$\text{code}(a_0a_1a_2\ldots a_n) = \sum_{i=0}^{n} \text{code}(a_i) \cdot m^i$$

$$= \text{code}(a_0) + \text{code}(a_1) \cdot m + \text{code}(a_2) \cdot m^2 + \ldots$$

$$= \text{code}(a_0) + m \cdot \underbrace{\left(\text{code}(a_1) + \text{code}(a_2) \cdot m + \ldots\right)}_{\text{code}(a_1a_2\ldots a_n)}$$

$$\text{code}(a_1a_2\ldots a_n) = \left(\text{code}(a_0a_1a_2\ldots a_n) - 1\right) / m$$

$$\text{code}(a_0) = \left(\text{code}(a_0a_1a_2\ldots a_n) - 1\right) \,\%\, m + 1$$

▸ Zugriff auf Buchstaben

$x \leftarrow a[i]$

```
x ← a
loop i
   --x
   x ← x / m
end
if x
   --x
   x ← x % m
   ++x
end
```

a[i] ← x

```
p ← m ^ i
c ← a[i]
c ← c · p
a ← a − c
c ← x · p
a ← a + c
```

▸ Länge des Strings

n ← len(a)

```
n ← 0
loop a
   c ← a[n]
   if c
      ++n
   end
end
```

▸ Ein- und Ausgabe

Konvention:
in = Eingabe
out = Ausgabe

return x

```
out ← x
stop
```

Entscheidungsproblemen:
out = *true* ⇒ akzeptiert
out = *false* ⇒ nicht akzeptiert

▸ Loop-Programm für $\{A^nB^nC^n \mid n \in \mathbb{N}_0\}$

```
n ← len(in)
nmod3 ← n % 3
if nmod3
    return false
end
i ← n / 3
j ← i + i
k ← n
loop i
    --i
    --j
    --k
    c ← in[i]
    if c ≠ code(A)
        return false
    end
    c ← in[j]
    if c ≠ code(B)
        return false
    end
    c ← in[k]
    if x5 ≠ code(C)
        return false
    end
end
return true
```

